Assignment 6: Medians and Order Statistics & Elementary Data Structures

Haeri Kyoung

University of the Cumberlands

MSCS-532-M20 – Algorithms and Data Structures

Professor Brandon Bass

July 4, 2025

Part 1: Implementation and Analysis of Selection Algorithms

Overview

This part explores two key algorithms for selecting the k-th smallest element in an array:

- Randomized Quickselect: Uses randomness to achieve expected linear time.
- Deterministic Selection (Median of Medians): Guarantees linear time in the worst case.

Both were implemented in Python, tested on various datasets, and compared through empirical and theoretical analysis.

1. Randomized Selection (Quickselect)

The randomized version follows the standard Quickselect strategy. A pivot is chosen randomly, and the array is partitioned. The algorithm recursively continues on the relevant half depending on the rank of the pivot.

- Time Complexity (Expected): O(n)
- Time Complexity (Worst-case): $O(n^2)$, though rare in practice
- Space Complexity: O(1) additional space (in-place)

Strengths:

- Very fast in average cases
- Simple and elegant implementation

Limitations:

- Performance may degrade with unlucky pivot selections
- Not suitable for scenarios requiring consistent worst-case performance

2. Deterministic Selection (Median of Medians)

The deterministic algorithm breaks the array into groups of 5, finds the median of each group, and recursively selects the median of those medians as the pivot. This ensures balanced partitions, avoiding worst-case degeneration.

- Time Complexity (Worst-case): O(n)
- Time Complexity (Best/Avg-case): O(n), though with more overhead than Quickselect
- Space Complexity: O(n) due to auxiliary lists

Strengths:

- Guarantees worst-case linear time
- Useful in time-sensitive applications where worst-case matters

Limitations:

- More overhead due to recursive median selection
- Slightly slower in practice for small arrays

3. Empirical Comparison

Tests were performed on arrays with varying sizes and distributions (random, sorted, reverse-sorted). Key observations:

- Randomized Select was consistently faster on random inputs.
- Deterministic Select was more stable, with fewer fluctuations in runtime.
- For small inputs (< 1000 elements), Quickselect was generally the better choice.
- For large or mission-critical inputs, Deterministic Select may be preferred despite the added complexity.

Part 2: Elementary Data Structures Implementation and Discussion

Overview

This part focused on implementing four fundamental data structures from scratch:

- Array
- Stack
- Queue
- Singly Linked List

Each structure was built using native Python lists or classes and tested for basic operations.

1. Array

- Operations: Insert, Delete, Access
- Time Complexity:
  - Insert: O(1) (append)
  - Delete: O(n) (due to shift)
  - Access: O(1)
- Use Cases: Random access, dense data storage, image pixels

2. Stack

- Implemented using a list (LIFO behavior)
- Operations: Push, Pop
- Time Complexity:
  - Push: O(1)
  - Pop: O(1)
- Use Cases: Function call management, backtracking, parsing expressions

3. Queue

- Implemented with a list (FIFO behavior)

- Operations: Enqueue, Dequeue
- Time Complexity:
  - Enqueue: O(1)
  - Dequeue: O(n) due to list shift
- Use Cases: Task scheduling, event queues, BFS traversal

Note: For more efficient dequeue, a deque from the collections module could be used.

4. Linked List

- Singly Linked List implementation with Node class
- Operations: Insert at head, Delete by value, Traverse
- Time Complexity:
  - Insert: O(1)
  - Delete: O(n)
  - Traverse: O(n)
- Use Cases: Dynamic memory use, insert-heavy workloads, undo/redo functionality

Discussion: Trade-offs and Applications

Arrays vs Linked Lists

Feature	Arrays	Linked Lists
Access	O(1)	O(n)
Insert/Delete	O(n) (middle)	O(1) (head)
Memory	Contiguous	Node-based (more flexible)

Linked lists are ideal for dynamic datasets where insertions and deletions are frequent, while arrays are better suited for scenarios requiring fast access and predictable size.

Stacks and Queues

While both can be implemented using arrays, their performance differs when dealing with large datasets. Stack operations are efficient with lists, but queues may suffer due to repeated shifting on dequeues.

Conclusion

This assignment strengthened both theoretical understanding and practical skills in algorithm design and data structures. The comparison of selection algorithms highlighted how different approaches solve the same problem under different constraints. Implementing core data structures from scratch provided valuable insight into their behavior, performance trade-offs, and appropriate use cases.

Whether building an efficient sort for large data or choosing the right structure for a system design, the lessons from this assignment are highly applicable in both academic and industry settings.